Databricks workspace and lakehouse basics



One platform runs BritMart's data pipeline



Notebooks and files

Hold the project work

Compute + Unity Catalog

Run code; govern tables and files

Delta tables + Lakeflow Jobs

Store Bronze, Silver, Gold; run tasks in order

Databricks is more than a notebook service

Managed data and AI platform built on lakehouse concepts

01

Ingest

Orders and stock

02

Transform

PySpark or SQL

03

Store

Governed Delta tables

> Schedule pipelines, monitor runs, expose clean tables to analysts and downstream systems.





Know the workspace areas

Workspace

Notebooks, files, folders

Catalog

Tables, schemas, volumes, permissions

Compute

Jobs & Pipelines

SQL

Git folders

> Code and small assets go in workspace files; datasets do not.

Keep notebooks easy to rerun

01

Parameters

02

Read inputs

03

Transform

04

Validate

05

Write outputs

Record metrics

> Move shared logic into Python modules; keep secrets outside the notebook.

Match compute to the workload

Serverless

Databricks manages the runtime

Jobs compute

Scheduled or triggered production tasks

All-purpose compute

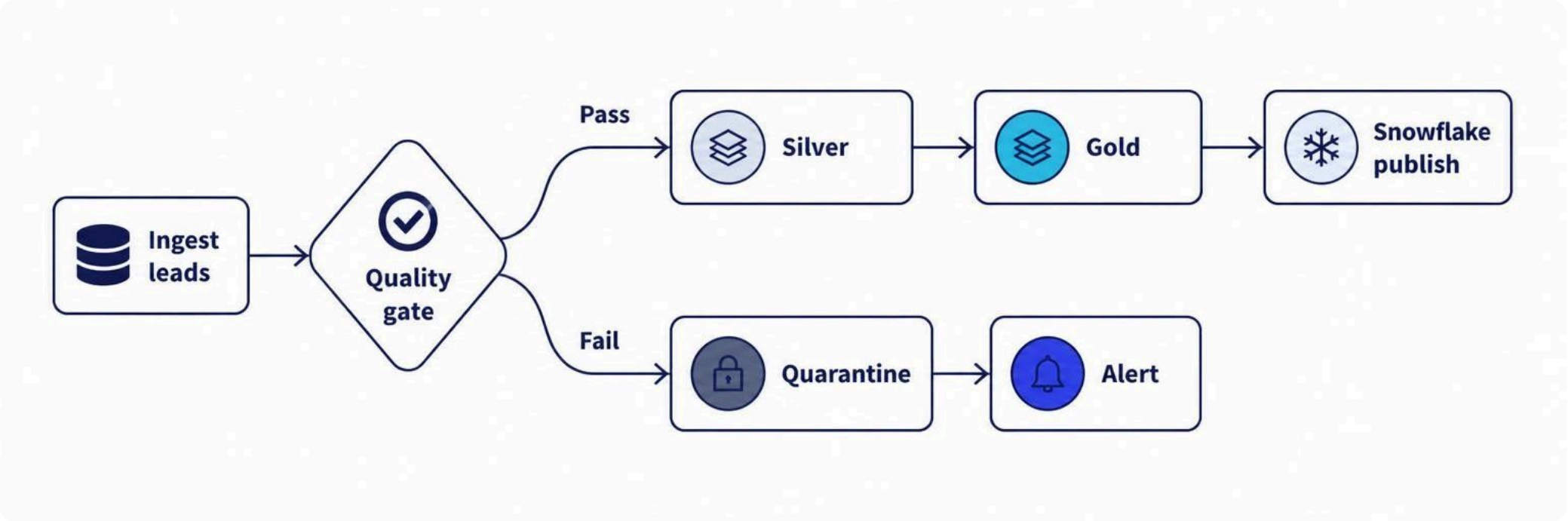
Interactive development

SQL warehouse

SQL, dashboards, BI



Lakeflow Jobs turns tasks into a pipeline



Triggers: manual, scheduled, file or table events, API

> Make tasks idempotent; add retries and failure notifications.

DBFS root is legacy: use Unity Catalog

DBFS root and mounts are deprecated, weakly governed.

Prefer volumes, external locations, managed tables

Volume path

```
/Volumes/<catalog>/<schema>/
<volume>/<path>
```





Pick the right storage for the data

Workspace files and Git folders

Code, config, small assets

Managed or external volume

Governed non-tabular files

Managed or external table

Governed tabular data

Read and write governed data

Tables

Read with

```
spark.table("main.bronze.orders")
```

Write reusable data with saveAsTable.

Volumes

Read files when file semantics matter.

Export files to a volume.

Directory writes create several part files.

> Never embed cloud keys.

Delta Lake adds a transaction log to data files



Data files

_delta_log

Committed table versions

Consistent reads, table history, `MERGE INTO`, time travel, schema checks

> Safer changes, but not a substitute for data quality checks.

Managed or external: who owns the storage?

Area	Managed table	External table
Storage location	Managed by Databricks/Unity Catalog	Specified external location
Common default	New governed tables	Shared or externally managed data
Drop behaviour	Managed lifecycle includes data	Metadata removed; files remain
Administration	Simpler central management	Requires external storage governance

> Managed: platform chooses location; drop removes data. External tables still need governed locations and credentials.

Bronze, Silver, Gold raise quality step by step

01

Bronze

Source records + ingestion metadata

Ready to replay

02

Silver

Reliable types, deduplication, key checks, reusable joins

03

Gold

Stable business tables

Daily sales by store



> A design pattern, not mandatory platform objects.

A complete workflow in seven steps

1. Ingest files into Bronze
2. Validate completeness
3. Bronze to Silver
4. Quality checks
5. Build Gold
6. Publish outputs
7. Record metrics, notify on failure

> Keep job definitions in version control.

Avoid these common mistakes

Teaching DBFS root as the default data lake

Treating Databricks as only notebooks; storing large data in workspace files

Secrets in code; interactive compute for every production job

Over-cleaning Bronze; mixing business aggregates into Silver

Assuming overwrite is safe; skipping retries and alerts

Design BritMart's lakehouse

Incoming and managed data

Volume for incoming partner files; managed Bronze, Silver, Gold tables

Partner-owned data

One external table for a partner-owned dataset

Four-task Lakeflow Job

Record-count and null-key gates

Run policy

Retry and failure notification policy; compute type noted per task

Check your understanding

1. Why is DBFS root no longer recommended for production?
2. Workspace file versus Unity Catalog volume?
3. When is a managed table a good default?
4. What does Delta Lake add to data files?
5. How do Bronze, Silver, and Gold differ?
6. What is a task dependency?
7. How does serverless change thinking about clusters?